Landon Hering April 28, 2025

6.1210 Problem Set 1

Problem 1 *(Collaborators: None)*

Notation: Let the sequence of integers $X = x_1, ...x_n$ be expressed as an array A that can be indexed ($A[0] = x_1, A[1] = x_2...$).

S Let $M(i, z, 0)$ be the length of the longest subsequence of $A[i :]$ that is increasing at $A[i]$ with z turns still allowed in the tail end of the sequence (without going over k turns). Similarly, let $M(i, z, 1)$ be the length of the longest subsequence of $A[i :]$ that is DECREASING at $A[i]$ with z turns.

R

$$M(i,z,0) = \max \begin{cases} M(i+1,z,0) & \text{skip element} \\ M(i+1,z-1,1)+1 & \text{Take element if } (A[i+1] \leq A[i]) \\ M(i+1,z,0)+1 & \text{Take element if } A[i+1] > A[i] \end{cases}$$

$$M(i,z,1) = \max \begin{cases} M(i+1,z,1) & \text{skip element} \\ M(i+1,z-1,0)+1 & \text{Take element if } A[i+1] \geq A[i] \\ M(i+1,z,1)+1 & \text{Take element if } A[i+1] < A[i] \end{cases}$$

These max functions represent 3 choices, skipping the element in the sequence (not including it in subsequence), taking the element to initiate a flip (goes from increasing to decreasing or decreasing to increasing), and taking the element while maintaining the state of increasing/decreasing.

T An entry $M(i, z, 0)$ or $M(i, z, 1)$ will always depend on entries with larger i. Thus the memo can be filled in reverse order from $i = n - 1$ to $i = 0$.

B The base cases are the cases where $i = n$ (this is beyond the sequence) in which case z can be anything from 0 to k, and then regardless of if it is increasing or decreasing the sequence length should be 0, so $M(n, z, 0) = M(n, z, 1) = 0$ for all $z \leq k$.

O The original subproblem is $\max(M(0, 0, 0), M(0, 1, 0), ...M(0, k, 0)$, which gives the length of the longest subsequence of A with up to k turns, starting strictly increasing.

T The runtime of this algorithm is $O(kn^2)$ because there are $O(kn)$ subproblems (i ranges from 0 to $n - 1$, and z ranges from k to 0, and the binary direction argument doesn't affect asymptotic runtime). Then, each of the subproblems can be solved in $O(1)$ time once inequalities are determined (all rely on previous subproblems that will be solved in constant time). Thus the overall runtime is $O(kn)$ $in(Okn^2)$ making this algorithm have an overall runtime of $O(kn^2)$.

Problem 2 *(Collaborators: None)*

S Let $M(i, j, k)$ represent the subproblem of the current characters of $A[i:], B[j:]$ and k consecutive letters in A. This problem can be solved by assigning a boolean value of True or False to each index in the DP table. Thus, let $M(i, j, k) = True$ if and only if $C[i + j:]$ can be produced by $A[i:]$ and $B[j:]$ when exactly k characters ($0 \leq k \leq 3$) of A have been taken consecutively most recently (and k never more than 3 in intermediate states).Let $m = len(A)$ and $n = len(B)$ for use in later parts.
R For each recurrence, we can either take an extra character from A, take one from B, or can do neither and must remain false (length of C is exactly the same as length of A plus length of B so can't skip any characters). Taking a character from the end of A can be done if $i > 0, k < 3, A[i-1] = C[i+j-1]$, and results in $M(i-1, j, k+1)$ being set to whatever $M(i, j, k)$ was. Taking a character from B can be done if $j > 0, B[j-1] = C[i+j-1]$, and results in $M(i, j-1, 0)$ being the value gotten by orring $M(i, j, k)$ for all k (regardless of the k this state must be true if $M(i, j, k)$ true). Nothing needs to be done in the case where neither of these can be done (this will be solved in base cases).
T It is clear that both i and j are decreasing separately. Thus, we can iterate i from the length of A down to 0, and for each value of i also iterate j down from the length of B down to 0. Then, for each i, j look at $k = 0, 1, 2, 3$ (this order is arbitrary because only when i or j decrease do the $k's$ matter) and because this is always decreasing i or j will get later states always processed first.
B Empty suffixes occur at $i = m$ and $j = n$, so need to set these $M(m, n, 0)$ to true. Everything else should be initialized as false, so then when going through the table the recurrence state will reassign these values to True if they are true, and keep them false if not true.
O The original problem is the result of $M(0, 0, k) = TRUE$ for any $k = 0, 1, 2, 3$, meaning that the characters from A and B can each be used exactly once with never more than 3 characters from A in a row. This original state could be viewed as starting with any number (0,1,2,3) consecutive characters from A, so if any of these k make it true, it should be true.
T There are $O(mn)$ subproblems because we iterate from $i = m$ to $i = 0$, and within each of these iterations of i we iterate j from $j = n$. Then we iterate a constant number (4) times for each value of k, but this constant factor does not affect overall runtime. Each of these subproblems takes $O(1)$ time because they are processed in a way that the later states are always processed first, so can be computed in linear time based on these later states. Thus, this results in an overall runtime of $O(nm)$.

Problem 3 *(Collaborators: None)*

Part 3(a)

S Every $M(i, j)$ will contain 2 key pieces of information, the maximum number of upgrades $U(i, j)$ that can be collected along a path from $i = 0, j = 0$ to the current square, along with the number of paths that reach that maximum number of upgrades from start to i, j (call this $P(i, j)$).
R First, get the number of upgrades from previous state u_n by setting $u_n = \max(U(i + 1, j), U(i, j + 1))$. Then, get the maximum number of upgrades for this state $U(i, j) = u_n + upgr(i, j)$, with $upgr(i, j) = 1$ if there is an upgrade on square i, j and being 0 if there is not an upgrade on the square. The number of paths $P(i, j) = P(i, j + 1) + P(i + 1, j)$ if both $U(i + 1, j) + upgr(i, j)$ and $U(i, j + 1) + upgr(i, j)$ equal the max number of upgrades $U(i, j)$. Otherwise, set $P(i, j) = P(i, j+1)$ if $U(i, j+1) > U(i+1, j)$ or $P(i, j) = P(i+1, j)$ otherwise. Finally, store both $U(i, j)$ and $P(i, j)$ in the memo ($M(i, j) = (U(i, j), P(i, j))$.
T The topological order is to start with $i = n - 1$ down to $i = 0$, and then for each i iterate j from $j = n - 1$ to 0. Because each subproblem only depends on those with larger i and larger j this will result in the later computations being computed in constant time based on previously computed subproblems.
B The base cases occur when a cell is a virus $M(i, j) = (-\infty, 0)$ and at $M(n - 1, n - 1) = (upgr(n - 1, n - 1), 1)$ where the first item in each tuple is $U(i, j)$ and the second is $P(i, j)$.
O The original problem is solved by doing $M(0, 0)$ which will give a tuple with the number of upgrades along the path and the number of ways to get such a path. Thus, you will return the number of ways to get such a path ($P(0, 0)$) and will be good.
T The algorithm will have $O(n^2)$ subproblems because it has to loop over all n values of j for each of the n values of i. Each of these subproblems can be solved in $O(1)$ time because the order they are solved results in a constant time operation with a couple conditionals for each of the previously computed subproblems (previously computed by topological order).